

Now, we construct a directed graph such that its vertices are tuples (n, m, k) and its edges connect from (n, m, k) to $(n, m, k + 1)$ or from (n, m, k) to (n', m', k') for $(n, m, k, n', m', k') \in C$. Then, for the constraints given in this paper, the graph is acyclic and the indegrees of vertices of the form $(n, m, 0)$ are zero. So we can compute the minimum values of $t_{n,m,k}$ satisfying (16), (17), and (20) by dynamic programming after the value of τ is determined.

Since the value of τ is not determined yet, we write $t_{n,m,k} = a_{n,m,k}\tau + b_{n,m,k}$ and define the order of pairs of $(a, b) \in \mathbb{Z} \times \mathbb{R}$ by the lexicographical order. Then, we can compute the minimum values of $a_{n,m,k}, b_{n,m,k}$ by dynamic programming satisfying the following conditions:

$$(a_{n,m,0}, b_{n,m,0}) = (0, 0), \quad (22)$$

$$(a_{n,m,k}, b_{n,m,k} + L_{n,m}) \leq (a_{n,m,k+1}, b_{n,m,k+1}), \quad (23)$$

$$(a_{n,m,k} + m, b_{n,m,k} + R) \leq (a_{n',m',k'} + m', b_{n',m',k'}) \quad (24)$$

for all $(n, m, k, n', m', k') \in C_1$.

We determine the values of $a_{n,m,k}, b_{n,m,k}$ as the minimum ones. Furthermore, we can prove that these values hold the following inequality by the forms of conditions:

$$a_{n,m,k} + m \leq M - 1. \quad (25)$$

Thus, when τ is large enough, $t_{n,m,k} = a_{n,m,k}\tau + b_{n,m,k}$ while satisfying the conditions (16), (17), (20), and (21). We determine the value of τ as the minimum one of such values.

C: Implementation Details

All the methods are implemented in C++ and evaluated with Intel Core i9-9900K CPU and 32GB RAM.

Optimization method. For optimization, we use Levenberg-Marquardt (LM) Algorithm implemented in g²o (?). Parameters for the optimization are set as the default values of the library.

We set $\sigma_r = \sigma_v = \sigma_o = \sigma_c = 10^4$ and $\sigma_t = 1$ as the initial values. Since the set C in Eq.(7) of the main paper may change depending on the value of p or $T_{i,j}$, we have to reset the explicit constraints repeatedly during optimization. The number of iteration of LM algorithm and intervals of resetting constraints are as follows:

1. The first 500 iterations: we recalculate C after each iteration.
2. The next 39500 iterations: we recalculate C for 10 iterations.
3. The following 1000 iterations: we recalculate C and increase $\sigma_r, \sigma_v, \sigma_o$, and σ_c by multiplying 1.01 and decrease σ_t by dividing 1.01 after each iteration.
4. The remaining iterations: we recalculate C after each iteration. This phase lasts until the plan converges, more precisely, until the difference of costs for 100 iterations goes below 10^{-6} .

Table 2 reports the number of iterations until the optimization converges and required times in minutes.

M	Environment					
	(a)	(b)	(c)	(d)	(e)	(f)
1	44537 2	42899 4	41113 5	45025 13	41133 29	46111 54
2	74280 15	97481 69	84898 59	49504 50	43514 69	73906 277
3	41910 8	83786 96	52978 57	85698 281	61067 188	41973 107

Table 2: Number of iterations (top) and required time [min] (bottom) for until the optimization converges for the proposed approach.

Baseline methods. For the FCFS approach, we modify the implementation of Prioritized SIPP by ? to our problem setting. For the SO approach, we modify the implementation of CCBS by the authors, which is available online³, to our problem setting. For each query, planning was considered a failure if the runtime exceeded 1.0 seconds. For PRM, We also use the implementation by ?. As the parameters for PRM, the number of roadmap vertices is 1000 for the FCFS approach and 20 for the SO approach. We limit the number of vertices for the SO because CCBS is generally time-consuming when the density of vertices is high and the environments are small. The number of neighbors is 15 for the FCFS approach and 10 for the SO approach.

Maxime veniam sapiente ea, quae repudiandae dolor a repellat commodi rem consectetur repellendus modi veritatis similique, cupiditate harum repellat dolores ullam alias a voluptate esse, distinctio commodi repellat culpa. Cupiditate accusamus magnam recusandae tempore nam soluta quas corrupti, voluptates quas est voluptate aut consequuntur autem voluptatum aperiam, ipsum rem illum dolor? Voluptatum deserunt nulla incident quae perferendis quaerat excepturi, ducimus autem dicta minima repellat magni provident animi enim nam debitis, assumenda unde perspiciatis sunt, nulla illum tenetur dignissimos. Qui nihil beatae ipsum incident, perferendis debitis repudiandae atque, dignissimos explicabo natus minima temporibus ipsa suscipit sequi cumque nulla ducimus veniam, quibusdam numquam tenetur ab ducimus? Earum cum officiis fuga placeat, similique ipsa fugit eius delectus impedit ut amet repellat quaerat dicta, ullam similique magni perferendis dolores provident, ut in porro corporis doloremque eos aliquid sint odit, saepe unde hic perferendis odit numquam perspiciatis voluptatibus ea sit delectus vero? Libero repellendus eveniet temporibus et quis accusantium eligendi, doloremque perspiciatis reiciendis consectetur officiis molestiae odit magni numquam architecto? Illum obcaecati adipisci reiciendis harum saepe sapiente dolorem ducimus reprehenderit quidem, quas quis quibusdam ipsa, voluptas excepturi voluptatum? Harum similique aperiam sed maxime illo ducimus, pariatur aperiam fuga totam eveniet corrupti repudiandae ab, consequatur velit sit minima exerci-

³github.com/PathPlanning/Continuous-CBS

tationem.Facilis corrupti praesentium quod doloremque as-
pernatur pariatur, qui fugiat quasi suscipit explicabo est mo-
lestiae quae excepturi cumque culpa dolorum,